SideQuest: A Social Chatting and Group Hangout Decision-Making Application

Week 3 Progress Report

Tazrean Tasneem, Tahmidatul Ayesha Asha, Zarifa Rahman, Tanzim Ahamad, Soumik Sikder

Course: CSE299 | Group: 02 | Section: 03

Abstract— The report summarizes the development of SideQuest in Week 3. SideQuest is a social chatting and group hangout decision-making application. The main objective was to move from previously integrated authentication workflow to persistent, real-time group and one-to-one communication. The group-chat functionality was developed by implementing the backend services, integrating them with the Flutter interface, and extending the group-management features to support a complete real-time communication experience.

A separate private-chat was also implemented using dedicated conversation and message models, REST(Representational State Transfer) endpoints, and an authenticated WebSocket channel. Both communication paths use PostgreSQL for message persistence. The application was executed in Google Chrome with separate user accounts to verify message exchange, sender identification, and retrieval of stored conversations.

Index Terms— Flutter, FastAPI, PostgreSQL, SQLAlchemy, WebSocket, JWT, group messaging, private chat, real-time communication.

I. INTRODUCTION

The developing phase of Week 2 established the connection among Flutter frontend, FastAPI backend, and PostgreSQL database, including registration, login, (JASON web token)JWT authorization, and database persistence. Based upon that foundation, the primary objective for Week 3 was to implement real-time communication. The work covered two communication modes: group conversations and one-to-one private conversations.

II. SYSTEM ARCHITECTURE

SideQuest uses Flutter for a client interface, FastAPI for HTTP and WebSocket services, and PostgreSQL for persistent storage. SQLAlchemy is used for database operations, while JWT tokens identify authenticated users. REST endpoints handle group and member operations, user search, conversation creation, and message-history retrieval. WebSocket connections carry live messages between connected participants. This separation allows the application to retrieve older messages through HTTP while using persistent socket connections for real-time delivery.

III. GROUP-CHAT BACKEND

The initial real-time group-chat backend introduces three SQLAlchemy models: Group, GroupMember, and ChatMessage, corresponding to the groups, group_members, and chat_messages tables. Group records store the group name and creator. Membership records associate users with groups and store the join timestamp. Chat-message records store the group ID, sender ID, message text, and creation timestamp.

The group router provides REST operations for creating groups, listing a user's groups, retrieving group details and members, adding or removing members, leaving a group, and deleting a group. A separate message-history endpoint retrieves messages in chronological order. These operations use JWT-based authentication and verify group membership before returning protected group data.

The central real-time component is the WebSocket endpoint "/ws/groups/{group_id}". Before accepting a connection, the backend validates the JWT, checks that the account is active, and confirms that the user belongs to the requested group. A connection manager tracks active WebSocket connections by group. When a client sends a non-empty message, the backend saves it to PostgreSQL and then broadcasts the saved message payload to connected clients in that group.

IV. FLUTTER GROUP-CHAT INTEGRATION AND MANAGEMENT

The group WebSocket functionality was connected to Flutter interface through a dedicated WebSocket service using the web_socket_channel package. The GroupChatScreen loads previous messages through the REST message-history endpoint, retrieves the group-member list, and establishes an authenticated WebSocket connection. Incoming messages are decoded and added to the conversation view; outgoing messages are sent through the active socket. The sender ID in each message is mapped to the corresponding username from the group-member list, so the interface displays member names rather than numeric user IDs.

Group creation is available to any authenticated user. When a group is created, the creator is automatically added as its Admin. The group list is refreshed after creation, and the newly created conversation can be opened directly. For member addition, as the Admin types a username prefix, the backend returns active users who are not already members, and Flutter interface displays matching names for selection. The selected user's database ID is then submitted to the add-member endpoint.

Group-management controls were added to the chat interface. The Admin can view members and remove a member, while any members can leave the group. If the Admin leaves, ownership is transferred to the oldest remaining member; if the Admin is the only member, the group and its messages are deleted. The backend also broadcasts membership and ownership events to connected clients and disconnects users who have left or been removed.

V. ONE-TO-ONE PRIVATE CHAT IMPLEMENTATION

A separate private-chat module was added along with the existing group-chat formulation. Two SQLAlchemy models were introduced: DirectConversation and DirectMessage, which create the direct_conversations and direct_messages tables. A direct conversation stores the two participant IDs and its creation timestamp. The participant IDs are stored in sorted order, with a uniqueness constraint on the pair, to prevent the same two users from creating duplicate conversations in opposite orders. Each direct message stores its conversation ID, sender ID, message text, and timestamp.

The private-chat REST endpoints support searching active users by username prefix, excluding the currently authenticated user; creating or reusing a conversation with another user; listing the current user's conversations; and

retrieving a conversation's message history. The backend checks that the requester is one of the two participants before returning private messages.

Real-time private messaging uses the authenticated WebSocket endpoint "/ws/private/{conversation_id}". The backend validates the JWT and verifies that the connected user is a participant in the requested conversation. A private connection manager tracks sockets by conversation ID. Each received message is checked for empty content and maximum length, saved to PostgreSQL, and then broadcast to the connected participants. The sender identity is taken from the authenticated token.

On Flutter side, the private-chat module includes a PrivateChatService, a PrivateChatScreen, and API-service methods for user search, conversation creation, conversation listing, and history retrieval. The screen loads stored messages, connects to the private WebSocket, displays incoming messages, and sends outgoing messages through the socket. The private-chat interface was integrated into the Chats area of the application.

VI. COMMUNICATION ENDPOINTS AND MESSAGE FLOW

The client uses REST requests to discover users, create or retrieve conversations, and load message history. After the history is loaded, the relevant WebSocket connection is opened with the user's JWT. For group chat, the socket is scoped to a group ID; for private chat, it is scoped to a direct-conversation ID. In both cases, the backend identifies the sender from the verified token, stores the message first, and broadcasts the stored message record. This keeps the database record and live event consistent.

VII. TESTING AND OBSERVED RESULTS

The application was tested in Google Chrome using separate authenticated user sessions. Both group and private messages were successfully exchanged in real time and stored in PostgreSQL. Previous messages were retrieved when conversations were reopened. The Flutter interface also supported group creation, username-based member search, member addition, and group management.

VIII. SECURITY AND ACCESS CONTROL

The application uses JWT authentication to ensure that only authorized users can access group and private conversations. Group members can access their respective group chats, while private conversations are restricted to the two participants. User search excludes inactive accounts and the currently logged-in user. The system also prevents duplicate private conversations and restricts access to message history. Additionally, blank messages are rejected, and private messages are limited to 5,000 characters. These security measures help protect user conversations and prevent unauthorized access.

IX. TESTING TOOLS AND APPROACH

Visual Studio Code was used for development, with FastAPI running through Uvicorn and Flutter in Google Chrome. The web_socket_channel package enabled WebSocket communication, while pgAdmin 4 was used to verify message storage in PostgreSQL. Both group and private chats were tested through separate authenticated browser sessions, confirming real-time message delivery and message-history retrieval through the Flutter interface.

XI. CURRENT PROGRESS AND FUTURE DEVELOPMENT

At the end of Week 3, SideQuest supports authenticated group and one-to-one conversations with real-time message delivery and PostgreSQL-backed message history. We collaboratively developed the group-chat functionality, covering backend implementation, Flutter integration, and group-management features such as group creation, username-based member search, member addition and removal, leaving groups, and ownership transfer. The private-chat module also supports user search, conversation creation, message exchange, and retrieval of previous messages through a dedicated WebSocket connection.

The next development stage will focus on developing the collaborative "Start a SideQuest" planning feature using the established communication system.

XII. LIMITATIONS AND NEXT STEPS

The current implementation maintains active WebSocket connections in backend application memory. Automatic WebSocket reconnection after temporary network interruptions has not yet been implemented. Additionally, reloading the Flutter application currently redirects users to the login screen because persistent authentication and automatic session restoration are not yet implemented. Future improvements will include securely retaining authentication credentials, restoring the user's session after a reload, and automatically reconnecting to the previous conversation. Messages sent during disconnection will remain stored in PostgreSQL and can be retrieved through the message-history endpoints when the connection is restored.

XIII. CONCLUSION

The development phase of Week 3 marks the transition of SideQuest from an authenticated application prototype to a working messaging application. The team-provided group WebSocket backend was integrated with Flutter, and the group interface was extended with member search and management features. A separate one-to-one private-chat implementation added conversation models, protected REST operations, WebSocket delivery, and persistent message storage. Functional checks in Chrome confirmed the main messaging workflows. These developments established the communication layer required for the next phase of collaborative hangout planning.